

Reviewer Pack — Minimal Rank of Tropicalized Boolean Functions over p-adic N...

Entry 8e5f17afb5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 18:30:26 UTC

Minimal Rank of Tropicalized Boolean Functions over p-adic Numbers vs AC0 Parity Circuit Size

Entry ID: 8e5f17afb5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 18:30:26 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Tropical Geometry) **Field B** (complexity object): Complexity Theory: AC0 Circuit Complexity

Statement:

[‘For any AC0 parity circuit C with n inputs, there exists a p -adically extended tropical polynomial $f(p^i)$ for each input i such that the minimal rank of f is $\Theta(\log(n))$ and the size of C is $\Theta(2^{O(\log(n))})$.’, ‘The minimal rank of the tropicalized Boolean function over p -adic numbers for an AC0 parity circuit is asymptotically equal to the logarithm of its size.’, ‘A counterexample to this conjecture would be a small AC0 parity circuit with a minimal rank that does not scale logarithmically with its size.’]

Rationale (proposer’s reasoning):

[‘Tropical geometry has been used to study complexity lower bounds, but applying it to p -adic numbers offers new algebraic structures that may reveal subtle properties of Boolean functions.’, ‘The logarithmic relationship between the minimal rank of a tropicalized function and the circuit size could provide insights into the structure of AC0 parity circuits.’, ‘This connection could potentially lead to new techniques for proving lower bounds in complexity theory.’]

Taxonomy category: AC0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e95b3a45efb4e741

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a parity circuit C , if the minimal rank of its tropicalized Boolean function over p -adic numbers is less than or equal to $\log(n)$ and the size of C is less than or equal to $2^{(0.5 \cdot \log(n))}$, it supports the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'tropical geometry' AND 'AC0 circuit complexity' AND p-adic numbers' - 'p-adic tropical polynomial' AND 'logarithmic rank' AND AC0 circuits' - 'AC0 parity circuit size' AND 'minimal rank' AND tropicalization

Top relevant hits considered: - [http://arxiv.org/abs/1206.1925v1] Counting Algebraic Curves with Tropical Geometry - [http://arxiv.org/abs/2201.08874v2] Note on p -adic Local Functional Equation - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/math-ph/0512018v2] On Phase Transitions for P -Adic Potts Model with Competing Interactions on a Cayley Tree - [http://arxiv.org/abs/1204.3875v2] Tropicalizing vs Compactifying the Torelli morphism - [http://arxiv.org/abs/2304.02770v2] Tight Correlation Bounds for Circuits Between AC0 and TC0 - [http://arxiv.org/abs/0705.3525v1] Comment on “Minimal size of a barchan dune” - [http://arxiv.org/abs/1110.2998v1] Equivalent Quantum Circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=4.7s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(matrix):
        rows, cols = len(matrix), len(matrix[0])
        for i in range(rows):
            max_row = i
            for j in range(i + 1, rows):
                if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                    max_row = j
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
            factor = matrix[i][i]
            for j in range(cols):
                matrix[i][j] /= factor
            for j in range(rows):
                if i != j:
                    factor = matrix[j][i]
                    for k in range(cols):
```

```

        matrix[j][k] -= factor * matrix[i][k]
    return matrix

def rank(matrix):
    rows, cols = len(matrix), len(matrix[0])
    rank = 0
    for i in range(min(rows, cols)):
        if matrix[i][i] != 0:
            rank += 1
    return rank

def generate_parity_circuit(n):
    circuit = []
    for _ in range(2**(n-1)):
        gate = random.choice(['AND', 'OR'])
        inputs = [random.choice([0, 1]) for _ in range(n)]
        circuit.append((gate, inputs))
    return circuit

def tropicalize_circuit(circuit):
    n = len(circuit[0][1])
    p_adic_matrix = [[0] * (n + 1) for _ in range(n + 1)]
    for gate, inputs in circuit:
        if gate == 'AND':
            for i in range(n):
                if inputs[i] == 1:
                    p_adic_matrix[0][i+1] += 1
        elif gate == 'OR':
            for i in range(n):
                if inputs[i] == 0:
                    p_adic_matrix[0][i+1] += 1
    return p_adic_matrix

def compute_minimal_rank(p_adic_matrix):
    rank_value = rank(gaussian_elimination(p_adic_matrix))
    return rank_value

n = random.choice([5, 10, 15, 20, 30, 40])
circuit = generate_parity_circuit(n)
p_adic_matrix = tropicalize_circuit(circuit)
minimal_rank = compute_minimal_rank(p_adic_matrix)

instances_tested = 1
conjecture_holds = minimal_rank == math.log2(n + 1)
counterexample = "" if conjecture_holds else "minimal_rank does not match log(n)"

return {
    "metric_name": "Minimal Rank",
    "metric_value": minimal_rank,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

```

```

results = []

for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {trial_result}")
    results.append(trial_result)

mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"minimal_rank does not match log(n)\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d8fab4ae.py", line 95, in <module>
    trial_result = run_trial(seed)
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d8fab4ae.py", line 76, in run_trial
    minimal_rank = compute_minimal_rank(p_adic_matrix)
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d8fab4ae.py", line 70, in compute_minimal_rank
    rank_value = rank(gaussian_elimination(p_adic_matrix))
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d8fab4ae.py", line 31, in gaussian_elimination
    matrix[i][j] /= factor
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means we cannot verify whether the conjecture's support conditions were met. | next: Re-run the test to ensure it completes without crashing and produces the necessary data for verification.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11245
2	preregistration	ollama_remote	glm4:latest	0	0	5500
3	novelty	ollama_remote	glm4:latest	0	0	4757
4	novelty	ollama_remote	glm4:latest	0	0	6140
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13218
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16866
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10700
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12316
9	judge	ollama_remote	glm4:latest	0	0	17150

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 97893 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/8e5f17afb5c5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8e5f17afb5c5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/8e5f17afb5c5.tar.gz` (if generated)